REVISION CHEAT SHEET:

TOPIC 17 OF 40

Structs, Records, and Enums in C#

Interview-ready notes · examples · C# · diagrams

Topic 17 of 40

C# gives you several different ways to model data:

EXAMPLE / FLOW

```
class

struct

record class

record struct

enum
```

They can sometimes look similar at first, but they have **very different semantics**.

Choosing the wrong one can lead to:

EXAMPLE / FLOW

```
Unexpected copying

Incorrect equality

Broken entity identity

Unnecessary allocations

Boxing

Mutable-value-type bugs

Poor EF Core modeling

Confusing APIs
```

The core decision is not:

ⓘ “Which syntax is shortest?”

It is:

ⓘ **What does this type mean, and what semantics should instances have?**

For your Fundraising Platform, examples might include:

EXAMPLE / FLOW

```
Gift
    →
Entity with identity
    →
```

```
class
```



```
GiftId
    →
Small value representing an ID
    →
readonly record struct

GiftSummaryDto
    →
Data-centric reference object
    →
record class

GiftStatus
    →
Finite set of named choices
    →
enum
```

High-Level Mental Model

EXAMPLE / FLOW

```
Need a type
    ↓
Does it represent
an entity/object identity?
    ↓
Usually class
```

EXAMPLE / FLOW

```
Does it represent
a small value
copied by value?
    ↓
struct / readonly struct
```

EXAMPLE / FLOW

```
Is it primarily data
and value equality is desired?
    ↓
record class
or
record struct
```

EXAMPLE / FLOW

```
Is it one choice
from a fixed named set?
    ↓
enum
```

Topic Coverage

This topic covers:

- `struct`;
- value semantics;
- struct assignment;

- struct method parameters;
- mutable struct problems;
- `readonly struct`;
- readonly members;
- struct constructors;
- parameterless struct constructors;
- `default`;
- struct primary constructors;
- boxing;
- struct interfaces;
- `ref struct`;
- class vs struct;
- records;
- `record`;
- `record class`;
- `record struct`;
- `readonly record struct`;
- positional records;
- compiler-generated properties;
- compiler-generated equality;
- `==`;
- `Equals`;
- `GetHashCode`;
- `ToString`;
- `Deconstruct`;
- `with`;
- shallow copying;
- record inheritance;
- records vs EF Core entities;
- enums;
- enum underlying types;
- enum values;
- zero/default enum values;
- validation;
- `[Flags]`;
- bitwise combinations;
- enums vs strings;
- ASP.NET Core usage;
- Fundraising examples;
- performance;
- security;
- common mistakes;
- interview questions.

Part 1 — Structs

What Is a Struct?

A `struct` is a **value type** that can contain data and behavior.

Example:

```C#
public struct GiftId
{
    public GiftId(
        long value)
    {
        Value = value;
    }

    public long Value
    {
        get;
    }
}
```

Usage:

```C#
GiftId id = new GiftId(1001);
```

A struct can contain:

EXAMPLE / FLOW

```
Fields

Properties

Methods

Constructors

Operators

Events

Indexers

Nested types

Interface implementations
```

Structs have **value semantics**: assigning or passing a struct by value copies the struct value.

Struct Is a Value Type

Consider:

```C#
public struct GiftId
{
    public long Value
    {
```

```
        get;
        set;
    }
}
```

Then:

C#

```
GiftId first = new GiftId { Value = 1001 };

GiftId second = first;
```

For a struct:

EXAMPLE / FLOW

```
first
    →
GiftId value

second
    →
Copied GiftId value
```

They are separate values.

Modify the Copy

C#

```
second.Value = 2002;
```

Now:

EXAMPLE / FLOW

```
first.Value
    =
1001

second.Value
    =
2002
```

Changing the copied struct doesn't change the original.

Compare with a Class

Class:

C#

```
public sealed class GiftReference
{
    public long Value
    {
        get;
        set;
    }
}
```

Then:

```C#
GiftReference first = new() { Value = 1001 };

GiftReference second = first;
```

Now:

```
first ──┐
        ↓
      Object
        ↑
second ─┘
```

Both variables reference the same object.

Struct Assignment vs Class Assignment

```
STRUCT
   →
Copy value
```

```
CLASS
   →
Copy reference
```

This is one of the most important distinctions in C#.

Value Type Does Not Mean "Always on the Stack"

A common myth is:

```
Struct
=
Stack

Class
=
Heap
```

That is not a reliable mental model.

A struct can exist:

```
As a local value

Inline inside another object

Inline inside an array

Inside another struct

Inside a boxed object
```

The important distinction is:

EXAMPLE / FLOW

```
Struct
    →
Value semantics

Class
    →
Reference semantics
```

not simply:

EXAMPLE / FLOW

```
Stack vs heap
```

Struct Parameters Are Copied by Default

C#

```
static void Change(
    GiftId id)
{
    id.Value = 9999;
}
```

Call:

C#

```
GiftId id = new() { Value = 1001 };

Change(
    id);
```

The method receives:

EXAMPLE / FLOW

```
a copy
```

so the original remains:

EXAMPLE / FLOW

```
1001
```

unless you explicitly pass by reference.

Struct and ref

C#

```
static void Change(
    ref GiftId id)
{
    id.Value = 9999;
}
```

Now:

```
C#
Change(
    ref id);
```

can modify the original variable.

This connects directly to Topic 5.

Struct Copying Cost

A small struct:

```
C#
public readonly struct GiftId
{
    public long Value
    {
        get;
    }
}
```

contains little data.

Copying it is inexpensive.

But imagine:

```
C#
public struct HugeStruct
{
    public decimal A;
    public decimal B;
    public decimal C;
    public decimal D;
    public decimal E;
    public decimal F;
}
```

Every by-value copy copies all of that state.

Microsoft specifically warns that passing or returning large structs by value can introduce copying costs; `in`, `ref`, `ref readonly`, and related mechanisms can avoid copies in performance-sensitive code.

When Structs Usually Work Well

Structs are most natural for values such as:

```
EXAMPLE / FLOW
Coordinates

Identifiers

Ranges

Dates/times

Small numeric concepts

Measurements

Small immutable value objects
```

Examples in .NET:

EXAMPLE / FLOW

```
int

decimal

DateTime

Guid

TimeSpan

DateOnly
```

are value types.

Struct Identity

Classes naturally support object/reference identity.

Two class objects may contain identical data but still be different objects.

Structs usually model:

EXAMPLE / FLOW

```
the value itself
```

rather than:

EXAMPLE / FLOW

```
which physical object instance is this?
```

That is why a small identifier wrapper can make sense as a struct.

Fundraising Example — GiftId

Instead of passing raw:

C#

```
long
```

everywhere:

C#

```
long giftId
```

a strongly typed identifier could be:

C#

```
public readonly record struct GiftId(
    long Value);
```

Then:

C#

```
GiftId giftId = new(1001);
```

This makes:

different conceptually and statically from:

EXAMPLE / FLOW

```
CampaignId

FundId

ConstituentId
```

if each receives its own type.

Why This Can Prevent Bugs

Without strong IDs:

C#

```
void Process(
    long giftId,
    long campaignId)
```

A caller could accidentally write:

C#

```
Process(
    campaignId,
    giftId);
```

The compiler sees:

EXAMPLE / FLOW

```
long

long
```

and accepts it.

With:

C#

```
void Process(
    GiftId giftId,
    CampaignId campaignId)
```

the types themselves carry domain meaning.

Struct Mutation

Structs **can** be mutable.

Example:

C#

```
public struct GiftCounter
{
    public int Count
    {
```

```
        get;
        set;
    }
}
```


But mutable structs are frequently problematic.

Microsoft recommends designing structs as immutable because copying mutable values can produce behavior that surprises callers.

Mutable Struct Problem

C#

```
GiftCounter first = new() { Count = 10 };

GiftCounter second = first;

second.Count++;
```

Now:

EXAMPLE / FLOW

```
first.Count
    =
10

second.Count
    =
11
```

Someone thinking about reference semantics might expect the first to change too.

It doesn't.

Mutable Struct in Collections

Mutable structs become especially confusing through collections/properties because values may be returned as copies.

For example, trying to mutate part of a struct returned through a collection indexer may produce compiler errors or change only a copy rather than the value you expected.

This is another reason the common recommendation is:

EXAMPLE / FLOW

```
Struct
    →
Prefer immutable
```

readonly struct

C# provides:

C#

```
readonly struct
```

to enforce stronger immutability guarantees.

Example:

```C#
public readonly struct GiftId
{
    public GiftId(
        long value)
    {
        if (value <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value));
        }

        Value = value;
    }

    public long Value
    {
        get;
    }
}
```

For a `readonly struct`, instance fields must be readonly, and properties must not expose ordinary mutation; `init`-only properties are permitted.

readonly record struct

If you also want generated record behavior:

```C#
public readonly record struct GiftId(
    long Value);
```

This provides:

EXAMPLE / FLOW

```
Value semantics

Record-generated equality

Generated ToString()

Generated Deconstruct()

with support
```

while preserving readonly struct semantics.

readonly Does Not Mean Deeply Immutable

Suppose:

```C#
public readonly struct Example
{
    public Example(
        List<int> values)
    {
        Values = values;
    }
```

```
    public List<int> Values
    {
        get;
    }
}
```

You can't replace:

EXAMPLE / FLOW

```
Values
```

through normal readonly struct mutation.

But:

C#

```
example.Values.Add(
    123);
```

can mutate the referenced `List<int>`.

Microsoft explicitly notes that a readonly struct containing a mutable reference type doesn't make the referenced object immutable.

readonly Struct Mental Model

EXAMPLE / FLOW

```
readonly struct
    →
Struct's own fields
cannot be reassigned through normal mutation
```

It does **not** automatically mean:

EXAMPLE / FLOW

```
Every object reachable
through those fields
is immutable
```

readonly Instance Members

You can also make individual struct members readonly:

C#

```
public struct Money
{
    private decimal _amount;

    public readonly decimal GetAmount()
    {
        return _amount;
    }
}
```

A readonly instance member promises not to mutate the struct's instance state.

This can also help the compiler avoid defensive copies in some scenarios.

Struct Constructors

Structs can have constructors.

Example:

```C#
public readonly struct GiftId
{
    public GiftId(
        long value)
    {
        if (value <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value));
        }

        Value = value;
    }

    public long Value
    {
        get;
    }
}
```

Usage:

```C#
GiftId id = new(1001);
```

Every Struct Has a Default Value

This is extremely important.

Even if your constructor rejects:

EXAMPLE / FLOW

```
0
```

this is still possible:

```C#
GiftId id = default;
```

Struct default state bypasses your normal validation constructor.

The runtime produces the default zeroed value.

Microsoft documents that `default(T)` for a struct ignores a user-defined parameterless constructor and produces the struct's default value; arrays of structs are also initially populated with default values.

Domain Consequence

Suppose:

```C#
public readonly struct GiftId
{
    public GiftId(
```

```
        long value)
    {
        if (value <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value));
        }

        Value = value;
    }

    public long Value
    {
        get;
    }
}
```

You might think:

EXAMPLE / FLOW

```
GiftId can never be zero
```

But:

C#

```
GiftId id = default;
```

produces:

EXAMPLE / FLOW

```
Value = 0
```

Therefore a struct cannot completely prevent its zero/default state from existing.

Important Struct Design Question

Before using a struct for a domain concept, ask:

EXAMPLE / FLOW

```
Does default(T)
make sense or at least remain safely recognizable?
```

For:

EXAMPLE / FLOW

```
GiftId
```

you may treat:

EXAMPLE / FLOW

```
Value == 0
```

as:

EXAMPLE / FLOW

```
Uninitialized / invalid
```

and validate at boundaries.

Parameterless Struct Constructors

Modern C# permits explicitly declared parameterless struct constructors.

Example:

```C#
public struct Example
{
    public Example()
    {
        Value = 10;
    }

    public int Value
    {
        get;
    }
}
```

Then:

```C#
Example value = new Example();
```

runs that constructor.

But:

```C#
Example value = default;
```

does not.

Likewise, an array:

```C#
Example[] values = new Example[10];
```

contains default-valued elements rather than invoking the parameterless constructor for every element.

new Struct() vs default

EXAMPLE / FLOW

```
new SomeStruct()
    →
May execute explicit
parameterless constructor
```

EXAMPLE / FLOW

```
default(SomeStruct)
    →
Zero/default state
```

They are not always equivalent in modern C#.

Struct Field Initialization

Modern C# allows struct field/property initializers subject to struct construction rules.

A struct constructed with `new` performs zero initialization, then applicable field initialization and constructor execution.

A struct produced with `default` gets its default bit-pattern state and ignores custom parameterless construction logic.

Struct Primary Constructors

Since C# 12, structs may declare primary constructors.

Example:

```C#
public readonly struct GiftId(
    long value)
{
    public long Value
    {
        get;
    } = value;
}
```

Important:

EXAMPLE / FLOW

```
value
```

does not automatically become a property merely because it appears in a normal struct primary constructor.

This follows the same normal-class primary-constructor model from Topic 12.

Struct Inheritance

A struct cannot derive from another class or struct in normal C# inheritance syntax.

This is invalid:

```C#
// public struct GiftId : SomeBaseClass
// {
// }
```

A struct also cannot be the base class for another type.

However:

EXAMPLE / FLOW

```
Struct
    →
Can implement interfaces
```

Microsoft documents these restrictions explicitly.

Struct Interfaces

Example:

```C#
public readonly struct GiftId :
    IComparable<GiftId>
```

```
{
    public GiftId(
        long value)
    {
        Value = value;
    }

    public long Value
    {
        get;
    }

    public int CompareTo(
        GiftId other)
    {
        return Value.CompareTo(
            other.Value);
    }
}
```

Structs can implement interfaces even though they can't participate in normal class inheritance.

Boxing Structs

Suppose:

C#

```
GiftId id = new(1001);

object boxed = id;
```

The struct value must be boxed into an object representation.

Likewise, converting a normal struct to an interface reference it implements may involve boxing.

Microsoft documents boxing conversions from ordinary struct types to `object`, `ValueType`, and implemented interfaces.

Boxing Cost

Boxing can involve:

EXAMPLE / FLOW

```
Heap allocation

Copying the value

Later garbage collection
```

So repeated boxing in hot code can hurt performance.

This connects to Topic 6.

Generics Often Avoid Boxing

Prefer:

C#

```
List<int>
```

instead of old non-generic:

```
C#
ArrayList
```

because generic containers can store values without boxing them to `object` in the normal case.

Struct Equality

Every struct gets `ValueType.Equals` behavior unless you provide another implementation.

But a normal custom struct does **not** automatically receive:

```
C#
operator ==
```

just because it is a struct.

Example:

```
C#
public struct GiftId
{
    public long Value;
}
```

You can use:

```
C#
first.Equals(
    second);
```

but:

```
C#
// first == second
```

doesn't automatically work unless equality operators are defined.

Why record struct Helps

A:

```
C#
record struct
```

gets compiler-generated equality infrastructure, including:

```
EXAMPLE / FLOW
Equals

GetHashCode

==

!=
```

which makes value-object modeling much more convenient.

ref struct

There is a specialized category:

```C#
ref struct
```

Example .NET types:

```EXAMPLE / FLOW
Span<T>

ReadOnlySpan<T>
```

A `ref struct` is restricted so that it can't escape to the managed heap.

Example:

```C#
public ref struct BufferView
{
    public Span<byte> Buffer;
}
```

`ref struct` types have special lifetime restrictions and are primarily intended for low-level/high-performance memory scenarios. C# 13 relaxed some restrictions—for example, ref structs may now implement interfaces under ref-safety rules—but they still cannot be boxed into those interface types.

Do You Need ref struct for Business Entities?

Usually:

```EXAMPLE / FLOW
No.
```

Do not make:

```EXAMPLE / FLOW
Gift

Campaign

Constituent
```

into ref structs.

Use `ref struct` for specialized:

```EXAMPLE / FLOW
Span-based

Parsing

Buffers

Memory-sensitive

High-performance
```

scenarios.

Choosing Class vs Struct

A practical heuristic:

Use a struct when the concept is:

EXAMPLE / FLOW

```
Small

Value-like

Preferably immutable

Frequently copied

Not identity-oriented
```

Use a class when:

EXAMPLE / FLOW

```
Object identity matters

State is larger

Shared mutation matters

Inheritance is needed

The object has a meaningful lifecycle
```

Microsoft's older framework design guidance recommends structs only for small, immutable, value-like types that won't be boxed frequently; although its exact byte-size guideline is dated, the semantic guidance remains useful.

Do Not Choose Struct Only to Avoid Heap Allocation

Bad reasoning:

EXAMPLE / FLOW

```
Struct is faster
because stack
```

Reality depends on:

EXAMPLE / FLOW

```
Size

Copy frequency

Boxing

Containing object

Array layout

JIT optimization

Usage pattern
```

A large struct copied thousands of times can perform worse than a small reference copy.

Choose semantics first.

Measure performance second.

Part 2 — Records

What Is a Record?

The:

```
C#
record
```

modifier tells C# to generate behavior especially useful for **data-centric types**.

A record can be either:

```
EXAMPLE / FLOW
Reference type
    →
record
record class
```

or:

```
EXAMPLE / FLOW
Value type
    →
record struct
```

Microsoft defines `record` / `record class` as reference types and `record struct` as a value type. Records add compiler-generated value equality, display formatting, and nondestructive mutation support.

Four Related Declarations

```
C#
public record GiftDto(...);
```

means:

```
EXAMPLE / FLOW
record class
```

This is also valid:

```
C#
public record class GiftDto(...);
```

Value-type version:

```
C#
public record struct GiftId(
    long Value);
```

Immutable value-style version:

```C#
public readonly record struct GiftId(
    long Value);
```

record Is Not Automatically a Struct

Very common interview question:

```C#
public record Person(...);
```

is a:

EXAMPLE / FLOW

```
Reference type
```

not a value type.

Only:

```C#
record struct
```

is a value type.

record class

Example:

```C#
public sealed record GiftSummaryDto(
    long GiftId,
    decimal Amount,
    string CurrencyCode);
```

This is a reference type.

Assignment:

```C#
GiftSummaryDto first = new(1001, 1000m, "USD");

GiftSummaryDto second = first;
```

copies:

EXAMPLE / FLOW

```
Reference
```

not the whole object.

But record class Has Value Equality

Even though it is a reference type:

```C#
var first = new GiftSummaryDto(1001, 1000m, "USD");

var second = new GiftSummaryDto(1001, 1000m, "USD");
```

then:

```C#
first == second
```

is:

EXAMPLE / FLOW

```
True
```

assuming all equality-participating values compare equal.

That is a defining difference from an ordinary class.

Ordinary Class Equality

Ordinary class:

```C#
public sealed class GiftSummary
{
    public long GiftId
    {
        get;
        init;
    }
}
```

Two separately constructed objects:

```C#
var first = new GiftSummary { GiftId = 1001 };

var second = new GiftSummary { GiftId = 1001 };
```

normally have:

EXAMPLE / FLOW

```
ReferenceEquals(first, second)
    =
False
```

and default class equality is based on object/reference identity unless overridden.

Record Equality

Record:

```C#
public sealed record GiftSummary(
    long GiftId);
```

Two separately created values:

```C#
var first = new GiftSummary(1001);

var second = new GiftSummary(1001);
```

have:

```
EXAMPLE / FLOW
first == second
    →
True
```

because record equality is value-oriented.

Records synthesize equality members including `Equals`, `GetHashCode`, `==`, and `!=`.

Value Equality vs Reference Equality

```
EXAMPLE / FLOW
Ordinary class
    →
Default identity/reference equality
```

```
EXAMPLE / FLOW
record class
    →
Reference storage semantics
+
Value equality
```

```
EXAMPLE / FLOW
normal struct
    →
Value storage/copy semantics
+
ValueType.Equals
```

```
EXAMPLE / FLOW
record struct
    →
Value storage/copy semantics
+
Compiler-generated value equality
+
== / !=
```

Why Records Are Useful

Records are excellent for:

```
EXAMPLE / FLOW
DTOs

Messages

Commands

Query results
```

```
Configuration data

Value objects

Snapshots

Immutable data models
```


where the **data values** matter more than object identity.

Positional Records

Concise syntax:

```C#
public sealed record GiftSummaryDto(
    long GiftId,
    decimal Amount,
    string CurrencyCode);
```

The compiler synthesizes public positional properties and a primary constructor.

For a `record class`, positional properties use `init`.

For a normal non-readonly `record struct`, positional properties are read/write.

For a `readonly record struct`, they are init-only.

Positional record class

```C#
public record ConstituentName(
    string FirstName,
    string LastName);
```

Conceptually supplies properties similar to:

```C#
public string FirstName
{
    get;
    init;
}

public string LastName
{
    get;
    init;
}
```

plus record-generated behavior.

Positional record struct

```C#
public record struct GiftCoordinate(
    int Batch,
    int Sequence);
```

By default, positional properties on a non-readonly record struct are mutable.

That surprises many developers.

Prefer readonly record struct for Immutable Values

```C#
public readonly record struct GiftId(
    long Value);
```

is often preferable to:

```C#
public record struct GiftId(
    long Value);
```

for identifier/value-object concepts.

Records Can Also Use Normal Property Syntax

You are not required to use positional syntax.

```C#
public sealed record GiftSummaryDto
{
    public required long GiftId
    {
        get;
        init;
    }

    public required decimal Amount
    {
        get;
        init;
    }

    public required string CurrencyCode
    {
        get;
        init;
    }
}
```

Records can contain normal:

EXAMPLE / FLOW

```
Properties

Methods

Fields

Constructors

Validation
```

just like other class/struct types.

Records Are Not Automatically Immutable

Very important.

This is valid:

```C#
public record GiftDto
{
    public decimal Amount
    {
        get;
        set;
    }
}
```

Records are optimized for immutable data modeling, but C# does not force all records to be immutable.

Record Primary Constructor vs Normal Primary Constructor

Normal class:

```C#
public class Gift(
    long giftId)
{
}
```

does not automatically create:

```EXAMPLE / FLOW
GiftId property
```

Record:

```C#
public record Gift(
    long GiftId);
```

does automatically synthesize a positional property.

Microsoft explicitly distinguishes record positional parameters from normal class/struct primary-constructor parameters.

Compiler-Generated Record Features

A record may receive generated support for:

```EXAMPLE / FLOW
Properties for positional parameters

Primary constructor

Equals

GetHashCode

==

!=
```

```
ToString

Deconstruct

with-copy semantics
```


depending on record form and declarations.

Record ToString

Example:

C#
```
public record GiftSummary(
    long GiftId,
    decimal Amount);
```

Then:

C#
```
Console.WriteLine(
    new GiftSummary(
        1001,
        500m));
```

produces a readable record-style representation containing the type and public property values.

This is often helpful during debugging and testing.

Record Deconstruction

Positional record:

C#
```
public record GiftSummary(
    long GiftId,
    decimal Amount);
```

can be deconstructed:

C#
```
var summary = new GiftSummary(1001, 500m);

var (
    giftId,
    amount) = summary;
```

Now:

EXAMPLE / FLOW
```
giftId = 1001

amount = 500
```

The compiler synthesizes a `Deconstruct` method for positional parameters.

with Expressions

One of the most useful record features is:

```C#
with
```

Example:

```C#
public sealed record GiftDraft(
    decimal Amount,
    string CurrencyCode);
```

Create:

```C#
GiftDraft original = new(1000m, "USD");
```

Create modified copy:

```C#
GiftDraft updated = original with { Amount = 1200m };
```

Now:

```EXAMPLE / FLOW
original.Amount
    =
1000

updated.Amount
    =
1200
```

A `with` expression creates a copy and applies the requested member changes.

Nondestructive Mutation

This pattern is called:

```EXAMPLE / FLOW
Nondestructive mutation
```

because:

```EXAMPLE / FLOW
Original object/value
    →
remains unchanged

New copy
    →
contains modifications
```

with Is Not Records-Only

Modern C# also supports `with` for:

where applicable.

It is not exclusively a record feature.

with Uses Shallow Copy Semantics

This is extremely important.

Suppose:

C#
```
public sealed record GiftDraft(
    decimal Amount,
    List<string> Notes);
```

Create:

C#
```
var notes = new List<string> { "Initial" };

var first = new GiftDraft(1000m, notes);

var second = first with { Amount = 1200m };
```

The record copy does not automatically deep-clone:

EXAMPLE / FLOW
```
Notes
```

Both records can still refer to the same list.

Shallow Copy Flow

EXAMPLE / FLOW
```
first  ──→ Notes List ←── second
```

So:

C#
```
second.Notes.Add(
    "Changed");
```

can affect what is seen through:

C#
```
first.Notes
```

too.

Record Does Not Guarantee Deep Immutability

A record:

```C#
public record GiftDraft(
    IReadOnlyList<string> Notes);
```

may expose a read-only interface, but the referenced object might still be mutable elsewhere.

Record-generated immutability features do not recursively freeze object graphs.

Record Equality Can Also Be Shallow

Consider:

```C#
public record Example(
    string[] Values);
```

Record-generated equality compares the member values using their equality semantics.

Arrays use reference-oriented equality rather than sequence equality.

Therefore two different arrays containing identical elements are not automatically equal merely because their contents match.

This is an important record equality trap.

Microsoft's record example demonstrates that records containing the same array reference remain equal even when elements of that shared array mutate.

Example

```C#
var first =
    new Example(
        new[]
        {
            "A",
            "B"
        });

var second =
    new Example(
        new[]
        {
            "A",
            "B"
        });
```

Even though contents look equal:

EXAMPLE / FLOW

```
["A", "B"]

["A", "B"]
```

the array references differ.

Do not automatically assume record equality means:

EXAMPLE / FLOW

```
deep structural equality
through every nested collection
```


Hash Codes and Mutability

Records synthesize:

EXAMPLE / FLOW

```
GetHashCode()
```

from their data.

If equality-participating mutable data changes after an object is inserted into:

EXAMPLE / FLOW

```
Dictionary

HashSet
```

you can break lookup behavior.

This is another reason records and value objects are often designed to be immutable.

Bad Mutable Key

C#

```
public record MutableKey
{
    public int Value
    {
        get;
        set;
    }
}
```

Suppose:

C#

```
var key = new MutableKey { Value = 1 };

var set = new HashSet<MutableKey> { key };
```

Then mutate:

C#

```
key.Value = 2;
```

The hash code may no longer correspond to the bucket where the object was stored.

Avoid mutable equality keys.

Record Class Inheritance

Record classes can participate in inheritance.

Example:

```C#
public abstract record TransactionResult(
    long TransactionId);
```

Derived:

```C#
public sealed record GiftResult(
    long TransactionId,
    decimal Amount)
    :
    TransactionResult(
        TransactionId);
```

Record-class equality also accounts for runtime record type, so two different derived record types with the same common values are not considered equal merely because their base data matches.

Record Inheritance Rules

A record class can inherit from another record class.

A normal class cannot derive from a record class, and a record class cannot derive from an ordinary user-defined class hierarchy in the same way.

Record structs don't support class inheritance.

record struct Inheritance

A:

```C#
record struct
```

is still a struct.

Therefore it:

```EXAMPLE / FLOW
Cannot derive from another struct/class

Cannot be a base class

Can implement interfaces
```

It does not participate in record-class inheritance hierarchies.

Record vs Entity

This distinction is critical for your Fundraising Platform.

Entity:

```EXAMPLE / FLOW
Gift
```

usually has identity:

```EXAMPLE / FLOW
GiftId = 1001
```

Two different object snapshots with the same values are not necessarily conceptually interchangeable merely because every property currently matches.

Entity modeling often depends on:

EXAMPLE / FLOW

```
Identity

Lifecycle

Change tracking

Reference identity
```

Records and EF Core Entities

Microsoft's record documentation specifically warns that EF Core relies on reference equality for entity tracking and therefore record types are not appropriate as EF Core entity types.

Therefore, for your persistence entities:

EXAMPLE / FLOW

```
Gift

Campaign

Fund

Constituent

GiftAllocation
```

prefer normal classes unless you have a very deliberate nonstandard design.

Good Record Uses in Fundraising

Records can still be excellent for:

EXAMPLE / FLOW

```
API request/response models

DTOs

Commands

Events/messages

Query projections

Value objects
```

Example:

C#

```
public sealed record GiftSummaryDto(
    long GiftId,
    string ConstituentName,
    decimal Amount,
    string CurrencyCode);
```

Record as Domain Value Object

Example:

```C#
public sealed record PostalAddress(
    string Line1,
    string City,
    string State,
    string PostalCode);
```

If addresses are treated as values:

```
EXAMPLE / FLOW
same fields
    →
same logical value
```

record semantics can fit naturally.

Whether this should be a record class or record struct depends on size, copying, mutability, and domain needs.

Record Class vs Record Struct

record class

```
EXAMPLE / FLOW
Reference type

Assignment copies reference

Value equality

Can inherit from other records

Good for larger data models / DTOs

GC-managed object
```

record struct

```
EXAMPLE / FLOW
Value type

Assignment copies entire value

Value equality

No class inheritance

Good for small value-like data
```

readonly record struct

Best suited for small immutable values:

```C#
public readonly record struct GiftId(
    long Value);
```

or:

```C#
public readonly record struct Percentage(
    decimal Value);
```

with proper validation added where needed.

Adding Validation to Record Struct

Simple positional syntax:

```C#
public readonly record struct GiftId(
    long Value);
```

doesn't automatically validate:

EXAMPLE / FLOW

```
Value > 0
```

You can use a fuller declaration/factory pattern when strong validation is required.

Example:

```C#
public readonly record struct GiftId
{
    public GiftId(
        long value)
    {
        if (value <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value));
        }

        Value = value;
    }

    public long Value
    {
        get;
    }
}
```

Remember default Again

Even this validated struct can still have:

```C#
GiftId id = default;
```

So design validation accordingly.

Part 3 — Enums

TOPIC 17 OF 40

What Is an Enum?

An `enum` represents a set of named integral constants.

Example:

```
C#
public enum GiftStatus
{
    Unknown,
    Pending,
    Completed,
    Refunded,
    Cancelled
}
```

An enum is a **value type**.

By default, enum members use `int` as the underlying type, beginning at zero and incrementing by one unless explicit values are assigned.

Why Use Enums?

Without enum:

```
C#
string status = "Completed";
```

Problems:

```
EXAMPLE / FLOW
"completed"

"Complete"

"COMPLETED"

"Compeleted"
```

are all possible strings.

An enum provides a finite named set:

```
C#
GiftStatus status = GiftStatus.Completed;
```

Now the compiler knows the type.

Enum Syntax

```
C#
public enum GiftStatus
{
    Unknown,
    Pending,
    Completed,
    Refunded,
```

```
    Cancelled
}
```


Equivalent numeric values are normally:

EXAMPLE / FLOW

```
Unknown
    =
0

Pending
    =
1

Completed
    =
2

Refunded
    =
3

Cancelled
    =
4
```

Explicit Enum Values

You can assign explicit numbers:

C#

```
public enum GiftStatus
{
    Unknown =
        0,

    Pending =
        10,

    Completed =
        20,

    Refunded =
        30,

    Cancelled =
        40
}
```

Explicit values can be useful when:

EXAMPLE / FLOW

```
Persisting stable external codes

Interoperating with protocols

Maintaining compatibility

Representing bit flags
```

Enum Underlying Types

The default underlying type is:

EXAMPLE / FLOW

```
int
```

You can choose another supported integral underlying type:

C#

```
public enum GiftStatus :
    byte
{
    Unknown =
        0,

    Pending =
        1,

    Completed =
        2
}
```

Supported underlying enum types include the standard integral forms such as:

EXAMPLE / FLOW

```
byte

sbyte

short

ushort

int

uint

long

ulong
```

Why Underlying Type Matters

A smaller underlying type can affect:

EXAMPLE / FLOW

```
Storage

Interop

Protocol compatibility
```

But don't choose:

C#

```
byte
```

merely for micro-optimization unless there's a concrete reason.

`int` is usually the simplest default.

Enum Default Value

Every enum's default value corresponds to numeric:

EXAMPLE / FLOW
```
0
```

even when no named member has value zero.

This is extremely important.

Dangerous Enum

C#
```
public enum GiftStatus
{
    Pending =
        1,

    Completed =
        2,

    Cancelled =
        3
}
```

Then:

C#
```
GiftStatus status = default;
```

produces numeric:

EXAMPLE / FLOW
```
0
```

but there is no named:

EXAMPLE / FLOW
```
GiftStatus.???
```

for zero.

Better

C#
```
public enum GiftStatus
{
    Unknown =
        0,

    Pending =
        1,

    Completed =
        2,
```

```
    Cancelled =
        3
}
```


Now default has meaningful representation:

EXAMPLE / FLOW

```
Unknown
```

Microsoft's enum guidance strongly recommends defining a zero value for ordinary enums and naming the zero value `None` for flag enums.

None vs Unknown

Use:

EXAMPLE / FLOW

```
Unknown
```

when zero means:

EXAMPLE / FLOW

```
Value hasn't been determined
```

Use:

EXAMPLE / FLOW

```
None
```

when zero means:

EXAMPLE / FLOW

```
No option selected
```

Example:

C#

```
public enum ReceiptDeliveryMethod
{
    None =
        0,

    Email =
        1,

    Mail =
        2
}
```

Choose semantics carefully.

Enum Values Are Not Strict Runtime Validation

This is valid:

C#

```
GiftStatus status = (GiftStatus)999;
```

even if:

EXAMPLE / FLOW

```
999
```

is not a declared member.

Enums do not automatically reject every numeric value that isn't explicitly named.

Validate Numeric Enum Input

C#

```
if (
    !Enum.IsDefined(
        status))
{
    throw new ArgumentOutOfRangeException(
        nameof(status));
}
```

Microsoft recommends `Enum.IsDefined` when validating values converted from numeric input.

Enum String Parsing

You can parse:

C#

```
GiftStatus status = Enum.Parse<GiftStatus>("Completed");
```

Safer external-input handling often uses:

C#

```
if (
    Enum.TryParse<GiftStatus>(
        input,
        ignoreCase: true,
        out GiftStatus status))
{
    // Valid parse syntactically.
}
```

But remember:

EXAMPLE / FLOW

```
Parsing numeric text
can potentially yield
an unnamed numeric enum value
```

so validation requirements depend on your input rules.

Enum.IsDefined

Example:

C#

```
GiftStatus value = (GiftStatus)500;

bool valid = Enum.IsDefined(value);
```

returns false when no member represents that value.

Enum Is a Value Type

Assignment:

```C#
GiftStatus first = GiftStatus.Pending;

GiftStatus second = first;
```

copies the enum value.

No object identity is involved.

Enums Are Stronger Than Magic Numbers

Bad:

```C#
if (
    gift.Status
    ==
    2)
```

What is:

EXAMPLE / FLOW

2?

Better:

```C#
if (
    gift.Status
    ==
    GiftStatus.Completed)
```

Meaning is visible immediately.

Enums Are Stronger Than Magic Strings

Bad:

```C#
if (
    gift.Status
    ==
    "completed")
```

Better:

```C#
if (
    gift.Status
    ==
    GiftStatus.Completed)
```

Benefits:

EXAMPLE / FLOW

```
Compiler checking

Refactoring support

Autocomplete

Clear finite choices
```


Enum Methods

You cannot normally define instance methods inside the enum declaration itself.

If behavior belongs near an enum, an extension method can be used.

Microsoft's enum documentation recommends extension functionality rather than defining methods directly in an enum declaration.

Enum Extension Method

C#

```
public static class GiftStatusExtensions
{
    public static bool IsFinal(
        this GiftStatus status)
    {
        return status
            is GiftStatus.Completed
            or GiftStatus.Refunded
            or GiftStatus.Cancelled;
    }
}
```

Usage:

C#

```
if (
    gift.Status.IsFinal())
{
}
```

But Don't Put Complex Business Logic in Enum Extensions

If status transitions become:

EXAMPLE / FLOW

```
Pending → Approved → Posted → Refunded

with authorization

dates

tenant rules

payment restrictions
```

that behavior probably belongs in:

EXAMPLE / FLOW

```
Gift

Gift workflow service

State model
```


not in a giant enum helper class.

Flags Enums

Sometimes multiple enum options can be combined.

Example:

C#

```
[Flags]
public enum GiftProcessingOptions
{
    None =
        0,

    Validate =
        1,

    SendReceipt =
        2,

    PublishEvent =
        4,

    All =
        Validate
        |
        SendReceipt
        |
        PublishEvent
}
```

For flag enums, individual values are normally powers of two so each option occupies a separate bit.

Binary View

EXAMPLE / FLOW

```
Validate
    =
001

SendReceipt
    =
010

PublishEvent
    =
100
```

Combine:

EXAMPLE / FLOW

```
Validate | SendReceipt
    =
011
```


Combine Flags

C#

```
GiftProcessingOptions options =
    GiftProcessingOptions.Validate
    |
    GiftProcessingOptions.SendReceipt;
```

Now both options are active.

Test a Flag

Bitwise:

C#

```
bool shouldSendReceipt =
    (
        options
        &
        GiftProcessingOptions.SendReceipt
    )
    == GiftProcessingOptions.SendReceipt;
```

Or:

C#

```
bool shouldSendReceipt = options.HasFlag(GiftProcessingOptions.SendReceipt);
```

Why Powers of Two?

Bad:

C#

```
[Flags]
public enum Options
{
    A =
        1,

    B =
        2,

    C =
        3
}
```

Because:

EXAMPLE / FLOW

```
3
=
1 | 2
```

`C` would overlap with:

EXAMPLE / FLOW

```
A + B
```

unless that composite meaning is intentional.

Correct Flags

C#

```
[Flags]
public enum Options
{
    None =
        0,

    A =
        1,

    B =
        2,

    C =
        4,

    D =
        8
}
```

None in Flags

For flags:

C#

```
None =
    0
```

means:

EXAMPLE / FLOW

```
No flags are set.
```

Microsoft's enum design guidance explicitly recommends the zero value `None` for flag enums.

When NOT to Use Flags

Don't create a flag enum if combinations are nonsensical.

Example:

EXAMPLE / FLOW

```
Gift Status:
Pending
Completed
Cancelled
```

A gift normally should not simultaneously be:

EXAMPLE / FLOW

```
Pending | Completed | Cancelled
```

So:

C#

```
[Flags]
GiftStatus
```

would be a poor design.

Simple mutually exclusive states should use a normal enum.

Enum vs Boolean

Suppose:

C#

```
bool isProcessed;
```

If only two permanent states exist:

EXAMPLE / FLOW

```
Yes / No
```

a bool may be enough.

But if state can evolve:

EXAMPLE / FLOW

```
Pending

Processing

Completed

Failed

Cancelled
```

an enum is clearer.

Enum vs Status Table

Not every status needs a database lookup table.

For stable application-defined states such as:

EXAMPLE / FLOW

```
Pending

Completed

Cancelled
```

a C# enum plus database constraint/mapping can be appropriate.

Use a separate database reference table when status values are:

EXAMPLE / FLOW

```
Business-configurable

User-maintained

Need metadata

Need localization

Need effective dates

Frequently changed without deployment
```

The choice should follow business semantics.

Enum vs String

Use enum when:

EXAMPLE / FLOW

```
Set is known

Set is finite

Application controls meanings

Compile-time checking is useful
```

Use string/reference data when:

EXAMPLE / FLOW

```
Values are externally defined

Values change dynamically

Customers configure values

Unknown future values must flow through unchanged
```

Persistence Warning

If you persist enum numeric values:

EXAMPLE / FLOW

```
Pending = 1

Completed = 2
```

then later reorder members without explicit values, database meaning may change.

Bad:

C#

```
public enum GiftStatus
{
    Pending,
    Completed
}
```

later:

```C#
public enum GiftStatus
{
    Draft,
    Pending,
    Completed
}
```

Numeric values shift.

Better for Persisted Numeric Enums

Use explicit stable values:

```C#
public enum GiftStatus
{
    Unknown =
        0,

    Pending =
        10,

    Completed =
        20,

    Refunded =
        30,

    Cancelled =
        40
}
```

Now adding:

```C#
Draft =
    5
```

doesn't change existing numeric meanings.

Do Not Reuse Old Numeric Enum Values

Suppose:

EXAMPLE / FLOW

```
20 = Completed
```

was historically persisted.

If `Completed` is later removed, don't assign:

EXAMPLE / FLOW

```
20 = Failed
```

to a new meaning.

Old persisted data could suddenly be interpreted incorrectly.

Enums and API Contracts

Enums are common in:

EXAMPLE / FLOW

```
Request DTOs

Response DTOs

Commands

Configuration
```

Example:

C#

```
public sealed record UpdateGiftStatusRequest(
    GiftStatus Status);
```

Whether JSON represents that enum as:

EXAMPLE / FLOW

```
number
```

or:

EXAMPLE / FLOW

```
string
```

is a serialization configuration/API-contract decision.

For public APIs, choose the representation deliberately and validate unknown values.

Class vs Struct vs Record vs Enum

Type	Semantics	Equality Default	Inheritance	Good For
`class`	Reference	Identity/reference	Yes	Entities/services
`struct`	Value	`ValueType.Equals`	No class inheritance	Small values
`record class`	Reference	Value equality	Record inheritance	DTOs/messages/data
`record struct`	Value	Value equality	No class inheritance	Small value records
`readonly record struct`	Value	Value equality	No class inheritance	Immutable small values
`enum`	Value	Numeric enum value	No class inheritance	Fixed named choices

Entity vs Value Object vs DTO vs Enum

Entity

EXAMPLE / FLOW

```
Gift
```

Identity matters:

EXAMPLE / FLOW
```
GiftId
```

Use:

EXAMPLE / FLOW
```
class
```

Value Object

EXAMPLE / FLOW
```
GiftId

Money

Address value

Percentage
```

Value equality often matters.

Possible choices:

EXAMPLE / FLOW
```
readonly record struct

record class
```

depending on size and copying.

DTO

EXAMPLE / FLOW
```
GiftSummaryDto
```

Mostly data.

Often good fit:

EXAMPLE / FLOW
```
record class
```

Fixed Choice

EXAMPLE / FLOW
```
GiftStatus
```

Use:

EXAMPLE / FLOW
```
enum
```

when choices are stable and application-defined.

Fundraising Example — Entity

C#

```
public sealed class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }
}
```

Why class?

EXAMPLE / FLOW

```
Identity

Lifecycle

EF Core tracking

Mutable controlled state
```

Fundraising Example — Value Identifier

C#

```
public readonly record struct GiftId(
    long Value);
```

Why?

EXAMPLE / FLOW

```
Small

Value-like

Strongly typed

Immutable

Value equality
```

Fundraising Example — DTO

C#

```
public sealed record GiftSummaryDto(
    long GiftId,
    string ConstituentName,
    decimal TotalAmount,
    string CurrencyCode,
    GiftStatus Status);
```

Why record class?

EXAMPLE / FLOW

```
Data-centric

Reference type

Value equality useful in tests

Concise

Readable ToString

with support
```

Fundraising Example — Enum

C#

```
public enum GiftStatus
{
    Unknown =
        0,

    Pending =
        10,

    Completed =
        20,

    Refunded =
        30,

    Cancelled =
        40
}
```

Why enum?

EXAMPLE / FLOW

```
Finite

Named

Mutually exclusive

Application-defined
```

Fundraising Example — Flags


```C#
[Flags]
public enum GiftProcessingOptions
{
    None =
        0,

    Validate =
        1,

    SendReceipt =
        2,

    PublishOutboxMessage =
        4,

    All =
        Validate
        |
        SendReceipt
        |
        PublishOutboxMessage
}
```

Usage:

```C#
GiftProcessingOptions options =
    GiftProcessingOptions.Validate
    |
    GiftProcessingOptions.SendReceipt;
```

Record with Example

```C#
GiftSummaryDto original =
    new(
        GiftId:
            1001,
        ConstituentName:
            "Alex Johnson",
        TotalAmount:
            1000m,
        CurrencyCode:
            "USD",
        Status:
            GiftStatus.Pending);

GiftSummaryDto completed = original with { Status = GiftStatus.Completed };
```

`original` remains unchanged.

Important Entity Warning

Do not do this merely because the syntax is attractive:

```C#
public record Gift(
    long GiftId,
    decimal TotalAmount);
```

for an EF Core tracked entity.

Value equality conflicts with the reference-identity semantics EF Core expects for entity instances.

ASP.NET Core Usage

A common API layering might look like:

EXAMPLE / FLOW

```
HTTP Request
    ↓
record request DTO
    ↓
Service
    ↓
class domain entity
    ↓
EF Core
```

Response:

EXAMPLE / FLOW

```
Entity / projection
    ↓
record response DTO
    ↓
JSON
```

Example Request Record

```C#
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal Amount,
    long? CampaignId);
```

This is concise and data-oriented.

Example Domain Entity

```C#
public sealed class Gift
{
    public Gift(
        long constituentId,
        decimal amount)
    {
        ConstituentId = constituentId;

        TotalAmount = amount;

        Status = GiftStatus.Pending;
    }
```

```
    public long ConstituentId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }
}
```

Different semantics for different responsibilities.

Example Response Record

C#

```
public sealed record GiftResponse(
    long GiftId,
    decimal Amount,
    GiftStatus Status);
```

Again:

EXAMPLE / FLOW

```
Data transfer
    →
Record fits naturally
```

Performance Considerations

Struct Copies

Large structs are copied during:

EXAMPLE / FLOW

```
Assignment

By-value method calls

By-value returns
```

unless optimizations/by-reference techniques apply.

Keep performance-sensitive structs reasonably small.

readonly Struct Optimization

`readonly struct` communicates immutability and can help the compiler avoid certain defensive copies.

Use it when value semantics and immutability fit the model.

Boxing

A struct may allocate when boxed to:

EXAMPLE / FLOW

```
object

ValueType

Interface
```

Frequent boxing can offset the allocation advantages you expected from using a struct.

Record Class Allocation

A:

EXAMPLE / FLOW

```
record class
```

is still a reference type.

Creating:

C#

```
new GiftSummaryDto(...)
```

normally creates an object just like another class.

Record syntax doesn't eliminate reference-type allocation.

record struct Copying

A:

EXAMPLE / FLOW

```
record struct
```

avoids reference-type semantics but copies the entire value on assignment.

So:

EXAMPLE / FLOW

```
record struct
```

is not automatically faster than:

EXAMPLE / FLOW

```
record class
```

for large data.

with Allocation/Copying

For record class:

EXAMPLE / FLOW

```
with
```

creates another record object and performs a copy.

For a value type:

EXAMPLE / FLOW

```
with
```

creates a modified value copy.

Do not use massive chains of `with` in hot code without understanding allocation/copy behavior.

Enum Performance

Enums are integral value types and are generally cheap.

Avoid converting them repeatedly to:

EXAMPLE / FLOW

```
string
```

in extremely hot paths if profiling shows it matters, but this is rarely a concern in ordinary ASP.NET Core business logic.

Security Considerations

Enums received from external input must still be validated.

Do not assume:

EXAMPLE / FLOW

```
GiftStatus parameter
```

means the runtime value must be one of your declared names.

Numeric conversions can produce unnamed values.

Example

C#

```
GiftStatus status = (GiftStatus)999;
```

Therefore at trust boundaries:

C#

```
if (
    !Enum.IsDefined(
        status))
{
    throw new ArgumentOutOfRangeException(
        nameof(status));
}
```

may be appropriate.

Enum Authorization Warning

Do not rely on user-submitted:

just because it maps to an enum.

Authorization roles/claims must come from trusted identity/authorization infrastructure.

An enum gives type safety—not trust.

Record Security

A record's convenient:

EXAMPLE / FLOW
```
ToString()
```

may include public property values.

Be careful when records contain:

EXAMPLE / FLOW
```
Passwords

Tokens

Secrets

PII

Payment information
```

and are automatically logged.

A readable generated `ToString()` can accidentally expose sensitive values in logs.

DTO with Sensitive Data

Example:

C#
```
public record LoginRequest(
    string UserName,
    string Password);
```

Avoid:

C#
```
_logger.LogInformation(
    "Request: {Request}",
    request);
```

if generated record formatting could expose password data.

with and Security-Sensitive State

Be careful with:

C#
```
user with
{
```

```
    IsAdministrator =
        true
}
```


for security models.

Convenient copying doesn't mean callers should be allowed to manufacture privileged state.

Use controlled authorization processes.

Common Mistake — Struct = Stack

Wrong:

EXAMPLE / FLOW

```
Struct always lives on stack.
```

Correct:

EXAMPLE / FLOW

```
Struct has value semantics.

Its storage location depends
on context.
```

Common Mistake — Struct Is Always Faster

Not true.

Large structs can incur expensive copying.

Boxing can allocate.

Mutable structs can cause defensive copies and difficult behavior.

Common Mistake — Mutable Struct for Entity

Bad candidate:

C#

```
public struct Gift
{
    public long GiftId;

    public decimal Amount;

    public GiftStatus Status;
}
```

Gift has:

EXAMPLE / FLOW

```
Identity

Lifecycle

Mutable business behavior

Persistence tracking
```

Use a class.

Common Mistake — Forgetting Struct Copy


```C#
GiftId second = first;
```

copies the value.

Changing `second` doesn't change `first`.

Common Mistake — Assuming readonly Means Deep Immutability

A readonly struct can still contain references to mutable objects.

`readonly` prevents reassignment of the struct's own state, not mutation of every referenced object.

Common Mistake — Constructor Makes Struct Default Impossible

Wrong.

Even if constructor validates:

EXAMPLE / FLOW

```
Value > 0
```

this always remains possible:

```C#
GiftId id = default;
```

Design around default values.

Common Mistake — new Struct() Always Equals default

Not necessarily.

A struct can define a parameterless constructor.

```C#
new MyStruct()
```

may execute it.

```C#
default(MyStruct)
```

doesn't.

Common Mistake — Struct Can Inherit Another Struct

No.

Structs can't participate in normal class inheritance hierarchies.

They can implement interfaces.

Common Mistake — Record Means Value Type

Wrong.

```
C#
record
```

and:

```
C#
record class
```

are reference types.

Only:

```
C#
record struct
```

is a value type.

Common Mistake — Record Means Immutable

Wrong.

Records can contain mutable properties.

Immutability is a design choice.

Common Mistake — Positional record struct Is Immutable

Wrong.

This:

```
C#
public record struct Point(
    int X,
    int Y);
```

generates mutable positional properties.

Use:

```
C#
public readonly record struct Point(
    int X,
    int Y);
```

for readonly semantics.

Common Mistake — Record Equality Is Deep Equality

Wrong.

Nested reference members use their own equality rules.

Lists/arrays aren't automatically compared element-by-element merely because the outer object is a record.

Common Mistake — with Deep Clones Everything

Wrong.

`with` is normally a shallow copy.

Reference-type members may still be shared.

Common Mistake — Record Is Ideal EF Entity

Usually wrong.

EF Core entity tracking relies on reference identity, and Microsoft specifically warns against using record types for EF Core entity types.

Common Mistake — Enum Guarantees Declared Value

Wrong.

```C#
GiftStatus status = (GiftStatus)12345;
```

is possible.

Validate untrusted numeric enum values.

Common Mistake — No Zero Enum Member

Bad:

```C#
public enum GiftStatus
{
    Pending =
        1,

    Completed =
        2
}
```

because:

```C#
default(GiftStatus)
```

still produces zero.

Prefer a meaningful zero member.

Common Mistake — Reordering Persisted Enum Members

If numeric values are persisted, changing declaration order can change meaning when values aren't explicit.

Use stable explicit values for externally persisted numeric contracts.

Common Mistake — Flags Values 1,2,3,4

Bad:

EXAMPLE / FLOW

```
1
2
3
4
```

because:

EXAMPLE / FLOW

```
3
```

already means:

EXAMPLE / FLOW

```
1 | 2
```

Use:

EXAMPLE / FLOW

```
1
2
4
8
16
...
```

for independent flags.

Common Mistake — Flags for Mutually Exclusive States

Don't use:

C#

```
[Flags]
GiftStatus
```

if a gift can only be in one status at a time.

Use ordinary enum.

Practical Exercise 1 — Struct Copying

C#

```
public struct Counter
{
    public int Value;
}
```

C#

```
Counter a = new() { Value = 10 };

Counter b = a;

b.Value = 20;
```

What is:

EXAMPLE / FLOW
```
a.Value?
```

Answer:

EXAMPLE / FLOW
```
10
```

Because:

EXAMPLE / FLOW
```
b
```

received a copy.

Practical Exercise 2 — Class Copying

C#
```
public sealed class Counter
{
    public int Value;
}
```

C#
```
Counter a = new() { Value = 10 };

Counter b = a;

b.Value = 20;
```

What is:

EXAMPLE / FLOW
```
a.Value?
```

Answer:

EXAMPLE / FLOW
```
20
```

because both references point to the same object.

Practical Exercise 3 — readonly record struct

Create:

C#
```
public readonly record struct FundId(
    long Value);
```

Then:

C#
```
FundId first = new(3001);

FundId second = new(3001);
```

Question:

```C#
first == second
```

Result:

EXAMPLE / FLOW

```
True
```

because record struct synthesizes value equality.

Practical Exercise 4 — Record Class Equality

```C#
public record GiftSummary(
    long GiftId,
    decimal Amount);
```

```C#
var first = new GiftSummary(1001, 500m);

var second = new GiftSummary(1001, 500m);
```

Results:

EXAMPLE / FLOW

```
ReferenceEquals(first, second)
    →
False

first == second
    →
True
```

Practical Exercise 5 — with

```C#
var original = new GiftSummary(1001, 500m);

var updated = original with { Amount = 750m };
```

What is original amount?

EXAMPLE / FLOW

```
500
```

Updated?

EXAMPLE / FLOW

```
750
```

Practical Exercise 6 — Enum Default

C#
```
public enum GiftStatus
{
    Pending =
        1,

    Completed =
        2
}
```

C#
```
GiftStatus status = default;
```

Numeric value:

EXAMPLE / FLOW
```
0
```

Even though no member represents zero.

Fix by adding:

C#
```
Unknown =
    0
```

Practical Exercise 7 — Flags

C#
```
[Flags]
public enum ProcessingOptions
{
    None =
        0,

    Validate =
        1,

    Notify =
        2,

    Publish =
        4
}
```

C#
```
ProcessingOptions options = ProcessingOptions.Validate | ProcessingOptions.Publish;
```

Binary:

EXAMPLE / FLOW
```
001
|
100
```

```
=
101
```


Practical Exercise 8 — Choose the Type

Gift entity

Choose:

EXAMPLE / FLOW

```
class
```

Why?

EXAMPLE / FLOW

```
Identity + lifecycle
```

GiftId

Choose:

EXAMPLE / FLOW

```
readonly record struct
```

Why?

EXAMPLE / FLOW

```
Small immutable value
```

GiftSummaryDto

Choose:

EXAMPLE / FLOW

```
record class
```

Why?

EXAMPLE / FLOW

```
Data-centric reference model
```

GiftStatus

Choose:

EXAMPLE / FLOW

```
enum
```

Why?

EXAMPLE / FLOW

```
Finite named state choices
```

Interview Questions

1. What is a struct in C#?

A struct is a value type that can encapsulate data and behavior.

Assignments and ordinary by-value method calls copy its value rather than copying an object reference.

2. What is the main difference between class and struct?

A class has reference semantics.

A struct has value semantics.

EXAMPLE / FLOW

```
Class assignment
    →
Copy reference

Struct assignment
    →
Copy value
```

3. Do structs always live on the stack?

No.

A struct may be stored as a local, inline inside an object, inside an array, inside another struct, or boxed.

Value/reference semantics are more important than the stack/heap simplification.

4. Can a struct contain methods?

Yes.

A struct can contain methods, properties, fields, constructors, operators, events, indexers, nested types, and more.

5. Can a struct inherit another class?

No.

A struct cannot derive from another class or struct, and can't be used as the base class of a class.

It can implement interfaces.

6. Why are immutable structs recommended?

Because structs are copied by value.

Mutation of copies can surprise callers and can be particularly confusing through properties and collections.

Immutable value semantics are easier to reason about.

7. What is a readonly struct?

A struct whose instance state can't normally be mutated after initialization.

Fields must be readonly and properties must be read-only or init-only as appropriate.

8. Does readonly struct guarantee deep immutability?

No.

A readonly struct can contain a reference to a mutable object.

The reference can't normally be reassigned, but the referenced object may still muta

9. What is the default value of a struct?

Its fields receive their default values—the zero/default state.

`default(T)` ignores custom parameterless struct constructor behavior.

10. Can a struct define a parameterless constructor?

Yes in modern C#.

`new S()` may invoke it, while `default(S)` does not.

11. Can a validated struct ever have an invalid default state?

Yes.

Even if every explicit constructor validates values, this remains possible:

```
C#
MyStruct value = default;
```

So default state must be considered in the design.

12. What is boxing with structs?

Converting a normal struct value to:

EXAMPLE / FLOW

```
object

ValueType

or an implemented interface
```

can box the value into an object representation.

This may allocate.

13. What is a ref struct?

A restricted struct whose instances can't escape into the normal managed heap.

Examples include:

EXAMPLE / FLOW

```
Span<T>

ReadOnlySpan<T>
```

They are intended primarily for memory/performance scenarios.

14. Can ref structs implement interfaces?

Beginning with C# 13, yes, under ref-safety restrictions.

However, they still can't be converted to an interface reference when that would require boxing.

15. What is a record in C#?

A record is a class or struct with compiler-generated data-oriented functionality such as:

EXAMPLE / FLOW

```
Value equality

ToString

with support

Deconstruction for positional records
```

TOPIC 17 OF 40

16. Is record a reference type?

Yes.

```
C#
record
```

and:

```
C#
record class
```

declare reference types.

```
C#
record struct
```

declares a value type.

17. What is record class?

A reference type with record-generated value-oriented behavior.

Assignment still copies references, but equality normally compares record data.

18. What is record struct?

A value type that also receives record-generated equality, formatting, and related features.

Assignments copy the value.

19. What is readonly record struct?

A record struct intended to be readonly/immutable at its own state level.

It's useful for small value objects and strongly typed identifiers.

20. Are records immutable?

Not necessarily.

Records can have ordinary mutable properties.

They are designed to support immutable data models conveniently, but immutability isn't mandatory.

21. What does positional record syntax generate?

For positional parameters, the compiler can generate:

EXAMPLE / FLOW

```
Properties

Primary constructor
```

plus normal record-generated behavior.

Property mutability depends on whether it is a record class, record struct, or readonly record struct.

22. What is the difference between a record primary constructor and a normal class primary constructor?

A positional record declaration synthesizes properties for its positional parameters.

A normal class/struct primary constructor doesn't automatically turn parameters into properties.

23. What is record value equality?

Two record instances are normally considered equal when they have compatible record runtime types and the equality-participating data members compare equal.

Records synthesize `Equals`, `GetHashCode`, `==`, and `!=`.

24. Does a normal struct automatically support ==?

Not necessarily.

A normal struct has `ValueType.Equals`, but a user-defined struct doesn't automatically receive custom `==` and `!=` operators.

A record struct does synthesize equality operators.

25. What does with do?

A `with` expression creates a copy of an existing value/object and applies selected changes.

It supports nondestructive mutation.

26. Is with a deep copy?

No.

Reference-type members may still point to the same objects.

Treat `with` as a shallow-copy mechanism unless your type explicitly implements deeper copying behavior.

27. Can with be used with structs?

Yes.

Modern C# supports `with` expressions for struct values as well as records and some other data forms.

28. Can record classes inherit?

Yes.

A record class can inherit another record class.

Record-class inheritance preserves record-specific equality semantics.

29. Can a normal class derive from a record?

No.

Record class inheritance must remain within the record class model rather than mixing ordinary class and record hierarchies.

30. Can record structs inherit?

No normal class inheritance.

A record struct follows struct inheritance rules and can implement interfaces.

31. Are records good EF Core entities?

Generally no.

Microsoft's record documentation notes that EF Core entity tracking depends on reference equality and therefore record types aren't appropriate entity types.

32. Where are records useful in ASP.NET Core?

Common examples:

EXAMPLE / FLOW

```
Request DTOs

Response DTOs

Commands

Events/messages

Query results

Immutable configuration/value models
```

33. What is an enum?

An enum is a value type representing a set of named integral constants.

34. What is the default underlying enum type?

EXAMPLE / FLOW

```
int
```

unless another supported integral type is specified.

35. What value do enum members start with by default?

The first member starts at:

EXAMPLE / FLOW

```
0
```

and later members increase by one unless explicit values are supplied.

36. What is default(EnumType)?

The zero-valued representation:

EXAMPLE / FLOW

```
(EnumType)0
```

even if no named zero member exists.

37. Why should most enums define a zero member?

Because:

```
C#
default(MyEnum)
```

always produces zero.

A meaningful zero member avoids unnamed default states.

38. Can an enum variable contain an undeclared numeric value?

Yes.

For example:

```
C#
GiftStatus status = (GiftStatus)999;
```

is possible.

Use validation when numeric values come from untrusted/external input.

39. What does Enum.IsDefined do?

It checks whether an enum defines a member corresponding to a particular value.

Useful for validating external numeric values.

40. What is a flags enum?

An enum whose values represent independent bits that can be combined.

Use:

```
C#
[Flags]
```

and power-of-two values.

41. Why use powers of two for flags?

Each option must occupy an independent bit:

EXAMPLE / FLOW

1

2

4

8

16

so bitwise combinations remain distinguishable.

42. Why should a flags enum have None = 0?

Because zero means:

```
EXAMPLE / FLOW
No flags set
```

and Microsoft specifically recommends `None` for that value.

43. Should GiftStatus be a flags enum?

Usually no.

Statuses such as:

```
EXAMPLE / FLOW
Pending

Completed

Cancelled
```

are mutually exclusive.

Use a normal enum.

44. What can happen if you reorder persisted enum members?

Their implicit numeric values can change.

Persisted records may then be interpreted with the wrong meaning.

Use explicit stable numeric values when numeric persistence is part of your contract.

45. When should you use enum instead of string?

When the set is:

```
EXAMPLE / FLOW
Finite

Known

Stable

Application-controlled
```

Use strings/reference data when values need to remain dynamic or externally defined.

Best Practices

Use:

```
EXAMPLE / FLOW
class
```

for domain entities where:

```
EXAMPLE / FLOW
Identity

Lifecycle

Reference semantics
```

matter.

For your Fundraising Platform, keep:

EXAMPLE / FLOW

```
Gift

Campaign

Fund

Constituent

GiftAllocation
```

as classes when they're EF Core entities.

Use:

EXAMPLE / FLOW

```
struct
```

only when value semantics genuinely fit.

Prefer structs that are:

EXAMPLE / FLOW

```
Small

Immutable

Value-oriented
```

Prefer:

C#

```
readonly struct
```

for immutable ordinary structs.

Prefer:

C#

```
readonly record struct
```

when you also want generated value equality and record conveniences.

Always think about:

C#

```
default(T)
```

when designing structs.

Do not assume constructors can make default struct state impossible.

Avoid large mutable structs.

Avoid unnecessary boxing.

Use `in` / `ref` techniques only when copying is actually a measured issue.

Use:

for data-centric reference models such as DTOs and messages.

Use records when:

EXAMPLE / FLOW
Value equality

matches the concept.

Do not choose records for identity-based EF entities simply for concise syntax.

Remember:

EXAMPLE / FLOW
record
≠
immutable automatically

Remember:

EXAMPLE / FLOW
with
≠
deep clone

Keep equality-participating record data immutable when records are used as dictionary/set keys.

Use enums for stable, finite, application-controlled choices.

Define a meaningful:

EXAMPLE / FLOW
0

member.

Use explicit numeric values when enum numbers are persisted externally.

Never assume all numeric enum values are valid.

Validate external enum data.

Use `[Flags]` only for values that can genuinely be combined.

Use powers of two for individual flags.

Use:

EXAMPLE / FLOW
None = 0

for flags enums.

Don't use flags for mutually exclusive states.

Do not use enums to represent user-configurable lookup data that must change without application deployment.

Be cautious logging records that contain sensitive data because generated `ToString()` is intentionally descriptive.

Quick Revision

Feature	class	struct	record class	record struct	enum
Reference/value	Reference	Value	Reference	Value	Value
Assignment	Copies reference	Copies value	Copies reference	Copies value	Copies value
Default equality	Reference identity	ValueType equality	Value equality	Value equality	Numeric value
`==` generated	Reference operator semantics	No custom structural `==` automatically	Yes	Yes	Yes
Inheritance	Yes	No class inheritance	Record inheritance	No	No
Interfaces	Yes	Yes	Yes	Yes	Interface use via Enum/ constraints, not normal declaration members
`with`	Not generally ordinary class	Yes for structs	Yes	Yes	No
`Deconstruct` generated	No	No	Positional records	Positional records	No
Best for	Entities/ services	Small values	DTOs/data models	Small data values	Fixed choices

Complete Fundraising Model

Strongly Typed Identifier

C#

```
public readonly record struct GiftId
{
    public GiftId(
        long value)
    {
        if (value <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(value));
        }

        Value = value;
    }

    public long Value
    {
        get;
    }

    public override string ToString()
    {
        return Value.ToString();
```

```
    }
}
```


CampaignId

```C#
public readonly record struct CampaignId(
    long Value);
```

FundId

```C#
public readonly record struct FundId(
    long Value);
```

The compiler now prevents accidental interchange in methods such as:

```C#
public void Assign(
    GiftId giftId,
    FundId fundId)
{
}
```

Gift Status

```C#
public enum GiftStatus
{
    Unknown =
        0,

    Pending =
        10,

    Completed =
        20,

    Refunded =
        30,

    Cancelled =
        40
}
```

Processing Options

```C#
[Flags]
public enum GiftProcessingOptions
{
    None =
        0,

    Validate =
        1,
```

```
    SendReceipt =
        2,

    PublishOutboxMessage =
        4,

    All =
        Validate
        |
        SendReceipt
        |
        PublishOutboxMessage
}
```

Gift Entity

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation>
        _allocations = [];

    public Gift(
        GiftId giftId,
        long tenantId,
        long constituentId,
        decimal amount)
    {
        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        GiftId = giftId;

        TenantId = tenantId;

        ConstituentId = constituentId;

        TotalAmount = amount;

        Status = GiftStatus.Pending;
    }

    public GiftId GiftId
    {
        get;
    }

    public long TenantId
```

```
    {
        get;
    }

    public long ConstituentId
    {
        get;
    }

    public decimal TotalAmount
    {
        get;
        private set;
    }

    public GiftStatus Status
    {
        get;
        private set;
    }

    public IReadOnlyList<GiftAllocation>
        Allocations =>
        _allocations;

    public void Complete()
    {
        if (
            Status
            !=
            GiftStatus.Pending)
        {
            throw new InvalidOperationException(
                "Only pending gifts can be completed.");
        }

        Status = GiftStatus.Completed;
    }
}
```

Why class?

EXAMPLE / FLOW

```
Gift
    →
Identity

Lifecycle

Mutability

Relationships

Persistence tracking
```

API Request Record

C#

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    decimal Amount,
    long? CampaignId);
```

Why record?

EXAMPLE / FLOW

```
Pure input data

Concise representation

No entity identity semantics needed
```

API Response Record

C#

```
public sealed record GiftResponse(
    long GiftId,
    decimal Amount,
    GiftStatus Status);
```

Query Projection Record

C#

```
public sealed record GiftSummaryDto(
    long GiftId,
    string ConstituentName,
    decimal Amount,
    string CurrencyCode,
    GiftStatus Status);
```

Copy DTO with with

C#

```
GiftSummaryDto pending =
    new(
        GiftId:
            1001,
        ConstituentName:
            "Jordan Smith",
        Amount:
            1000m,
        CurrencyCode:
            "USD",
        Status:
            GiftStatus.Pending);

GiftSummaryDto completed = pending with { Status = GiftStatus.Completed };
```

Now:

EXAMPLE / FLOW

```
pending.Status
    →
Pending

completed.Status
    →
Completed
```

Processing Flags


```C#
GiftProcessingOptions options =
    GiftProcessingOptions.Validate
    |
    GiftProcessingOptions.SendReceipt;
```

Check:

```C#
if (
    options.HasFlag(
        GiftProcessingOptions.Validate))
{
    // Validate gift.
}
```

```C#
if (
    options.HasFlag(
        GiftProcessingOptions.SendReceipt))
{
    // Send receipt.
}
```

No:

```EXAMPLE / FLOW
PublishOutboxMessage
```

flag is set in this example.

Type Selection Flow

```EXAMPLE / FLOW
Need to model something
        ↓
Does object identity matter?
```

Yes:

```EXAMPLE / FLOW
class
```

Example:

```EXAMPLE / FLOW
Gift
Campaign
Constituent
Fund
```

No:

EXAMPLE / FLOW

```
Does it represent
one small value?
```

Yes:

EXAMPLE / FLOW

```
readonly struct
or
readonly record struct
```

Example:

EXAMPLE / FLOW

```
GiftId
FundId
```

Otherwise:

EXAMPLE / FLOW

```
Is it mainly data
with value equality?
```

Yes:

EXAMPLE / FLOW

```
record class
```

Example:

EXAMPLE / FLOW

```
GiftSummaryDto
CreateGiftRequest
```

Otherwise:

EXAMPLE / FLOW

```
Is it a finite
set of named choices?
```

Yes:

EXAMPLE / FLOW

```
enum
```

Example:

EXAMPLE / FLOW

```
GiftStatus
```

Record Selection Flow

EXAMPLE / FLOW

```
Need record behavior
    ↓
```

```
Should variable assignment
copy reference or entire value?
```


Reference:

EXAMPLE / FLOW

```
record class
```

Value:

EXAMPLE / FLOW

```
record struct
```

Then:

EXAMPLE / FLOW

```
Should record struct
be immutable?
```

Usually yes:

EXAMPLE / FLOW

```
readonly record struct
```

for small value objects.

Enum Selection Flow

EXAMPLE / FLOW

```
Need named choices
    ↓
Can exactly one be active?
```

Yes:

EXAMPLE / FLOW

```
Normal enum
```

Example:

EXAMPLE / FLOW

```
GiftStatus
```

No:

EXAMPLE / FLOW

```
Can multiple independent
options be combined?
```

Yes:

EXAMPLE / FLOW

```
[Flags] enum
```

Example:

Complete Semantic Flow

EXAMPLE / FLOW

```
class
    →
Reference semantics
    →
Identity-oriented objects
```

EXAMPLE / FLOW

```
struct
    →
Value semantics
    →
Small values
```

EXAMPLE / FLOW

```
record class
    →
Reference semantics
+
Value equality
    →
Data-centric reference models
```

EXAMPLE / FLOW

```
record struct
    →
Value semantics
+
Record-generated equality
    →
Small value-oriented data models
```

EXAMPLE / FLOW

```
enum
    →
Named integral value
    →
Finite choices
```

Final Mental Model

Do not ask:

EXAMPLE / FLOW

```
Which syntax is shortest?
```

Ask:

EXAMPLE / FLOW

```
What should assignment mean?
```

```
What should equality mean?

Does identity matter?

Can this value be copied cheaply?

Should state be immutable?

Should inheritance be possible?

Is this an entity?

Is this a value?

Is this merely data?

Is this a finite choice?
```


Then choose the type.

Final End-to-End Fundraising Model

EXAMPLE / FLOW

```
Gift
    →
class
    →
Entity identity
```

EXAMPLE / FLOW

```
GiftId
    →
readonly record struct
    →
Small immutable value
```

EXAMPLE / FLOW

```
GiftSummaryDto
    →
record class
    →
Data-centric API/query model
```

EXAMPLE / FLOW

```
GiftStatus
    →
enum
    →
One finite state
```

EXAMPLE / FLOW

```
GiftProcessingOptions
    →
[Flags] enum
    →
Multiple combinable options
```

The most important lesson from this topic is:

Classes represent reference-oriented objects, structs represent value-oriented, records add value-oriented equality and data-centric compiler support, and enums represent named finite choices. Choose based on semantics—especially identity, equality, copying, mutability, and lifetime—not merely syntax or assumptions about stack versus heap performance.

For your Fundraising Platform, a strong default is:

EXAMPLE / FLOW

```
EF Core entities
    →
class

Small strongly typed IDs/value objects
    →
readonly record struct

API DTOs / commands / responses
    →
record class when value semantics fit

Stable application statuses/types
    →
enum
```

This completes the **OOP phase** of the roadmap.

Next roadmap topic: Topic 18 — Exception Handling.